COL1101 Lecture 18 2026/09/02

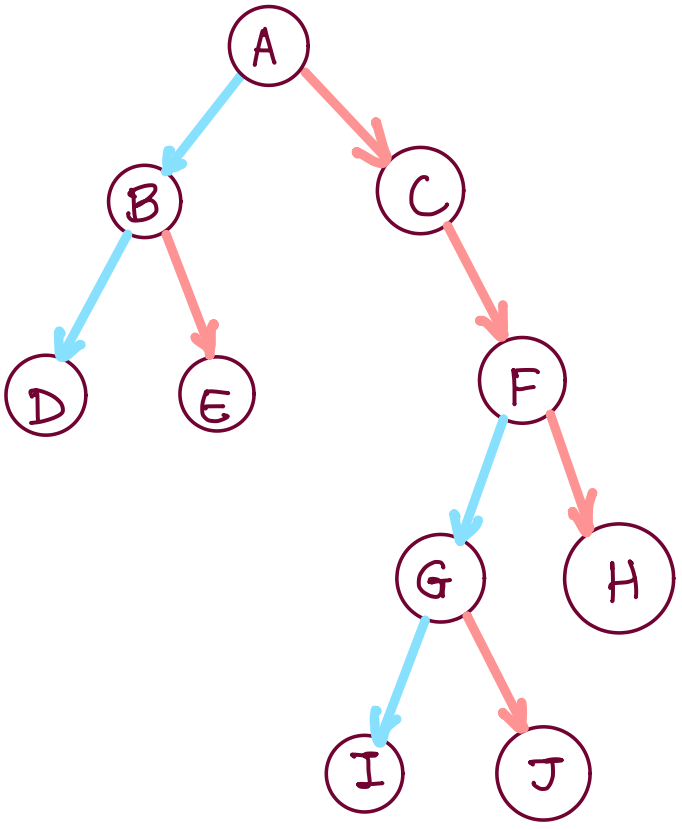
Topic: Almost Complete Binary Trees

Announcements

Exam OS Testing today (optional)

Bharti 501 13:00 – 17:00

Recap



binary tree

nodes, edges,

parent, left child, right child

ancestors, descendants

subtrees

depth, height

post-/pre-/in- order traversals

Go-to idea for algorithm design on trees: recursion

Implementation of Binary Trees

class BinaryTreeNode:

Attributes: key, left, right, parent

left, right, parent: references to BinaryTreeNode objects.

class BinaryTree:

Attribute: root → reference to a BinaryTreeNode object

Claim: Let n be the # nodes and h be the height of a binary tree. Then

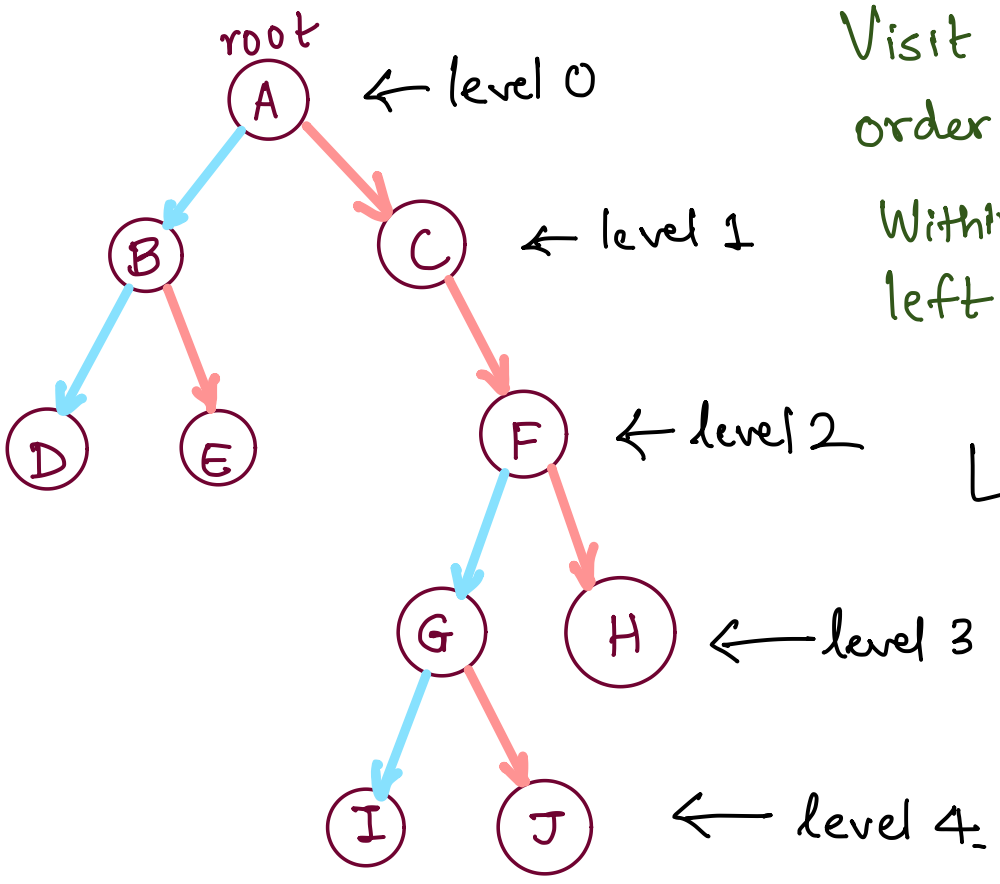
$$\log_2(n+1) \leq h \leq n \leq 2^h - 1$$

(the two middle inequalities are equivalent.)

Proof: There are h levels: $0, 1, \ldots, h-1$. In each level ℓ, we have $\leq 2^\ell$ nodes $\therefore n \leq 1 + 2 + \cdots + 2^{h-1}$

$= 2^h - 1$

Level-Order Traversal (LOT)



Visit nodes in increasing order of levels

Within each level from left to right.

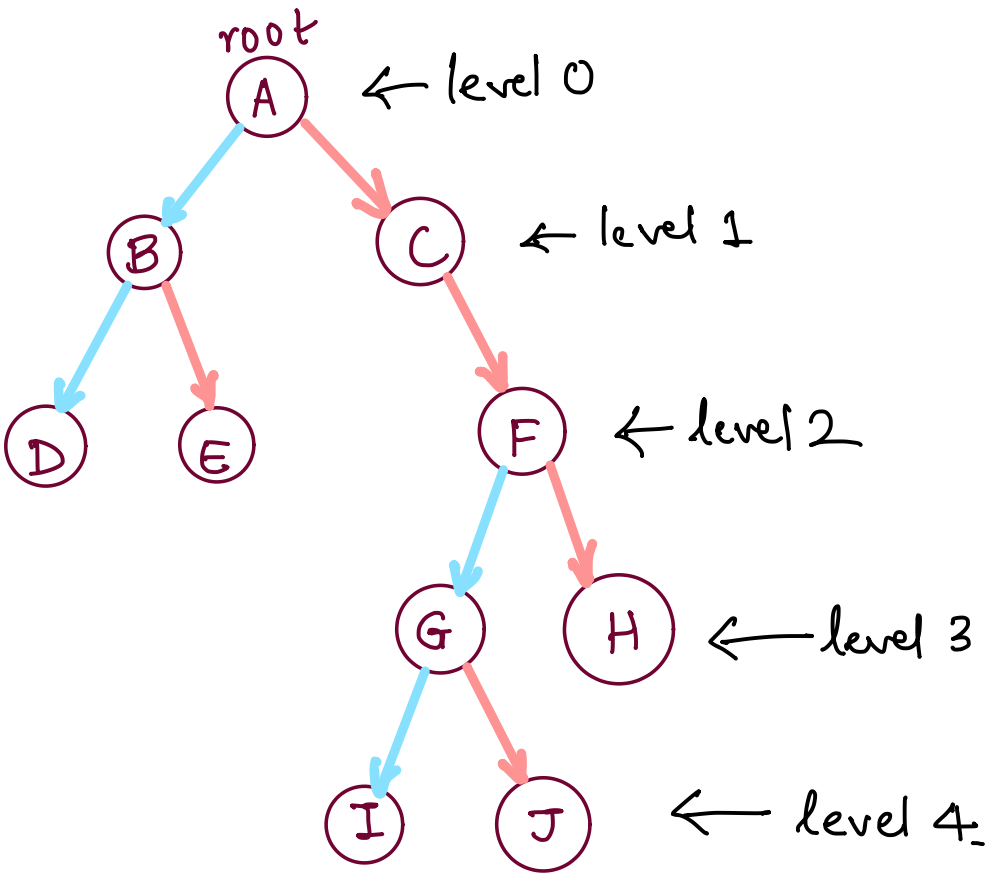
Level-order traversal:

A B C D E F G H I J

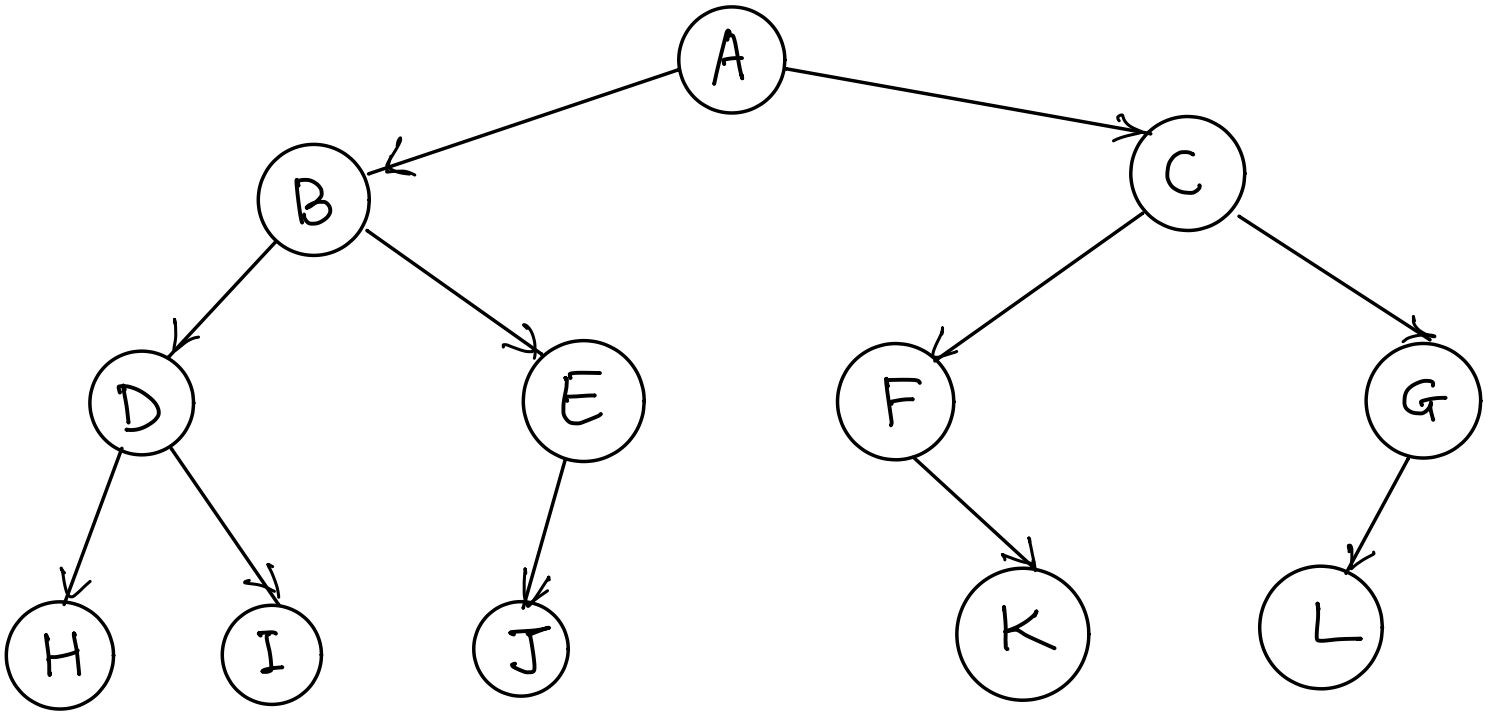
0 1 2 3 4

Level-Order Traversal Algorithm

```
q ← empty queue
q.enqueue(root)
While q is not empty:
    x ← q.dequeue().
    Visit x
    If x.left is not None
        q.enqueue(x.left)
    If x.right is not None
        q.enqueue(x.right)
```



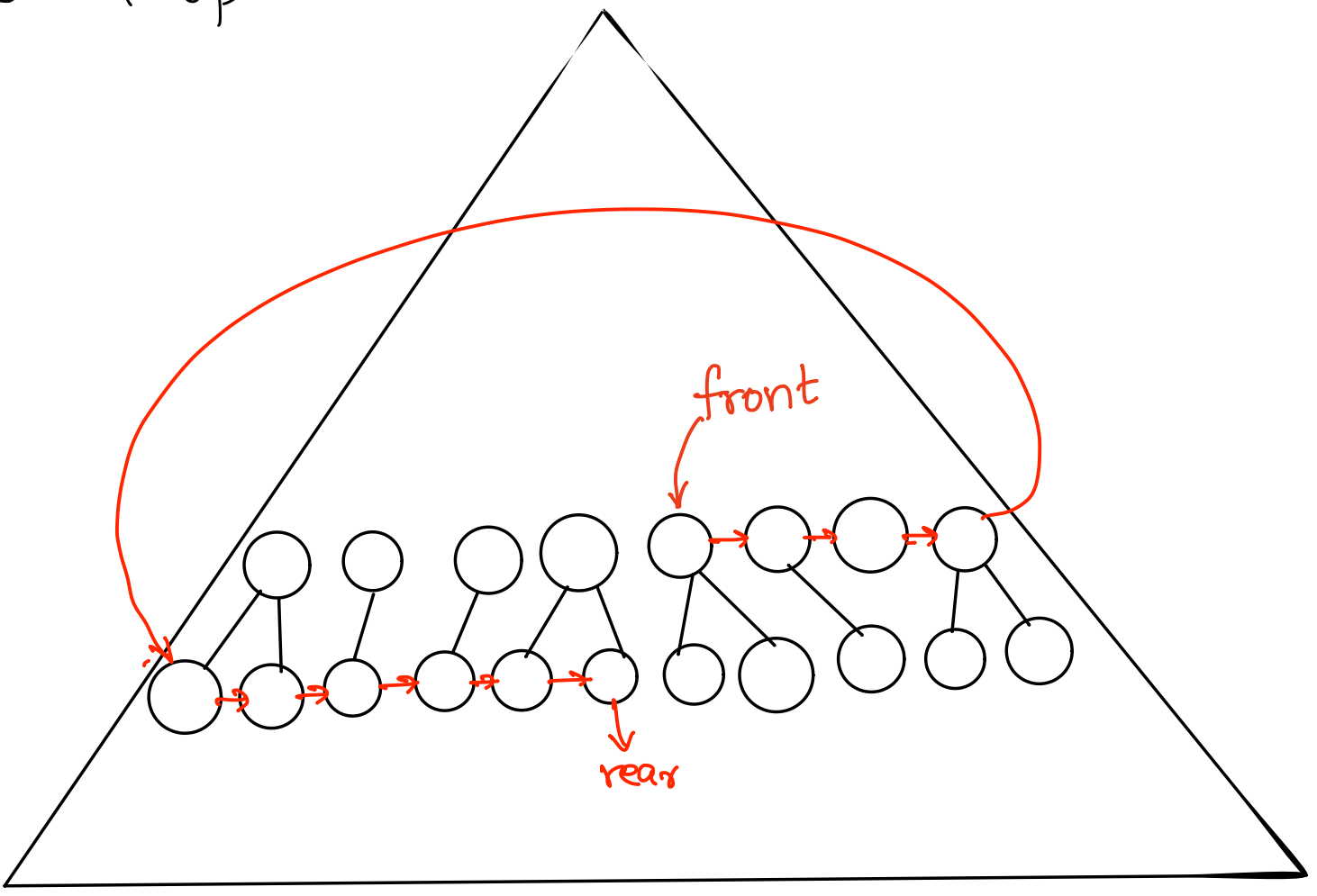
Visited node	Queue
	A
A	BC
B	CDE
C	DEF
D	EF
E	F
F	GH
G	HIJ
H	IJ
I	J
J	empty.



Question: When F is in the front of the queue, what does the queue contain?

Answer: F G H I J

LOT loop invariant



Claim: Suppose node x is in the front of the queue. Let d be the depth of x. Then the queue contains:

- x, followed by
- Nodes in level d to the right of x, in a left-to-right order, followed by.
- Children of (nodes in level d to the left of x) in a left-to-right order.

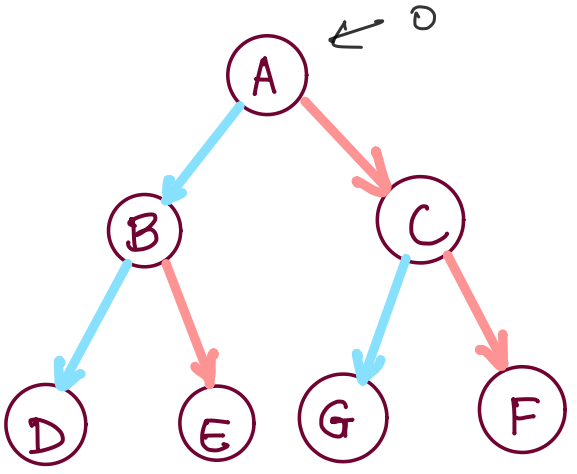
Proof: By induction on the # iterations of the while loop in the LOT algorithm.

Complete Binary Tree (CBT)

Complete binary tree of height h:

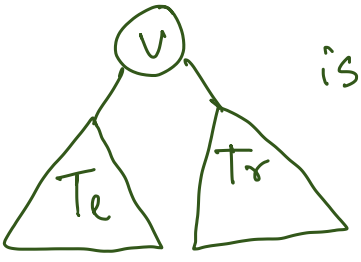
- All nodes in levels 0 ..... h-2 have two children each.
- All nodes in level h-1 are leaves.

root (Equivalently: tree with h full levels.)



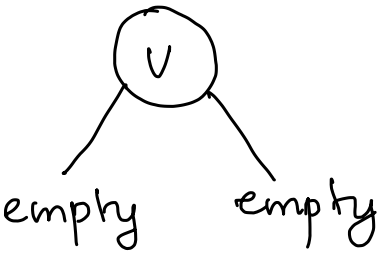
nodes in a complete binary tree of height h $= 2^h - 1$.

Complete Binary Tree: recursive definition

- The empty tree is the CBT of height 0.
- 

is a CBT of height h if T_l, T_r are both CBTs of height h-1.

eg. for h = 1



i.e. 

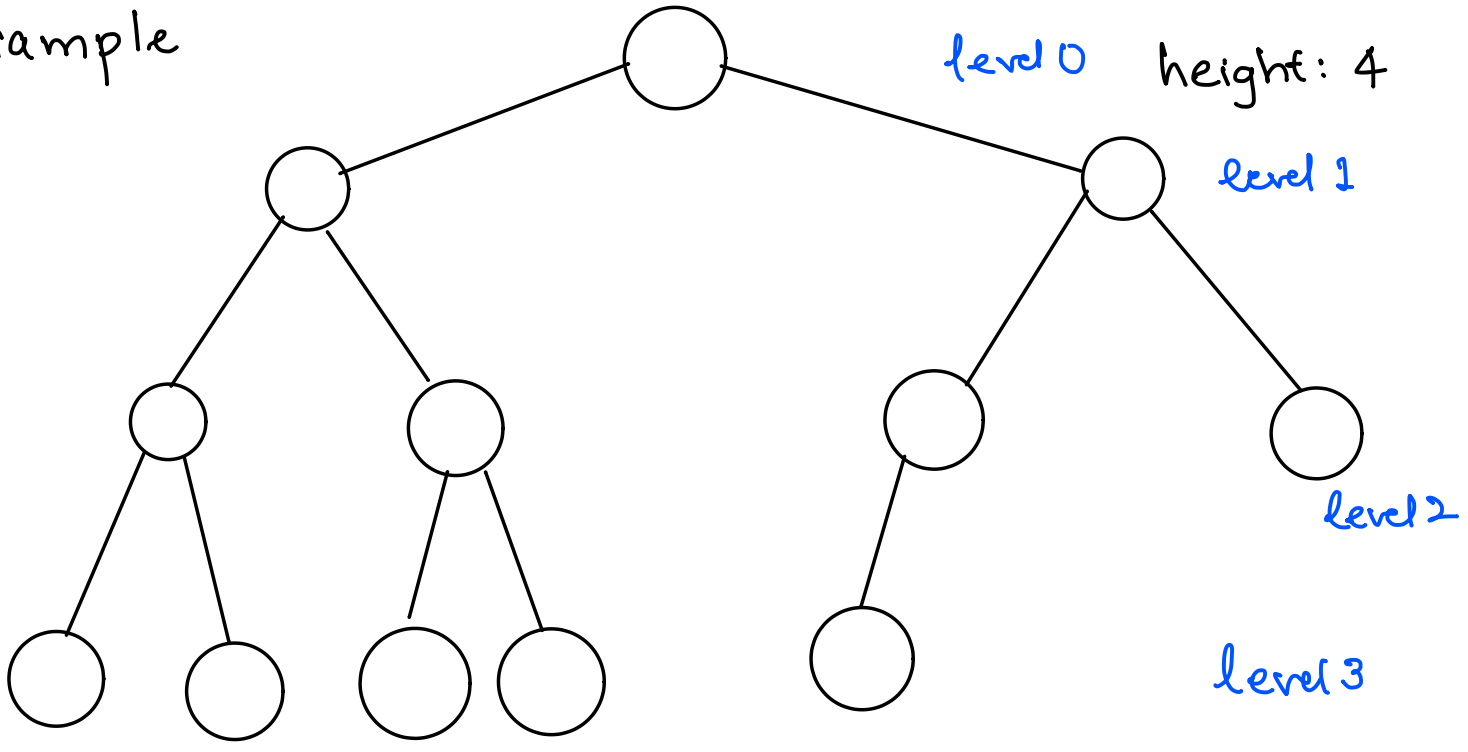
Almost-Complete Binary Trees (ACBT)

Definition :

An almost-complete binary tree of height h is one in which

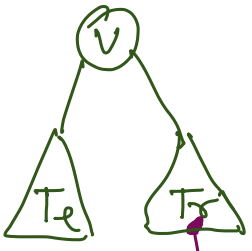
- Each level $\ell \in \{0, \ldots, h-2\}$ has 2^{ℓ} nodes (ie. it is full).
- Nodes in level $h-1$ (all leaves) are as much to the left as possible.

Example



All complete binary trees are almost-complete binary trees.

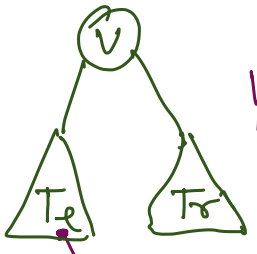
Almost Complete Binary Tree: recursive definition

- The empty tree is the ACBT of height 0.
- 

is an ACBT of height h if
  - T_ℓ is a CBT of height h-1 and T_r is an ACBT of height h-1.

  OR

  - T_ℓ is an ACBT of height h-1 and T_r is a CBT of height h-2.



rightmost leaf in bottommost level

rightmost leaf in bottommost level

Claim: Let n be the # of nodes in an almost complete binary tree and h be its height. Then $2^{h-1} \le n \le 2^h - 1$.

Proof: Each level ℓ contains $\le 2^\ell$ nodes. Sum up over $\ell = 0, \ldots, h-1$

Bottommost level $(h-1)$ contains ≥ 1 node. Each of levels $\ell = 0 \ldots h-2$ contains exactly 2^ℓ nodes.

$$n \ge \underbrace{1}_{\text{level 0}} + \underbrace{2}_{\text{level 1}} + \cdots + \underbrace{2^{h-2}}_{\text{level } h-2} + \underbrace{1}_{\text{level } h-1} = 2^{h-1}$$